UNIVERSITÀ DEGLI STUDI DI MILANO

AMD Project - Finding Similar Abstracts

AUTHOR

Luca Codeluppi - 32289A

Contents

1 Introduction

This project implements a scalable duplicate-detection system for scientific abstracts in the arXiv dataset. Detecting near-duplicate or very similar abstracts can be useful for several tasks: spotting proceedings versions of the same article, identifying multiple submissions of the same paper to adjacent categories and flagging plagiarism. The combinatorial cost of comparing all pairs makes a brute-force approach basically impossible: with N documents one has $\binom{N}{2} = O(N^2)$ pairs to compare, which is unmanageable.

The proposed solution combines a Spark ML preprocessing pipeline with the classical MinHash + LSH approach, achieving near-linear scaling on the number of documents while keeping high recall on near-duplicate pairs. The end-to-end system consists of a tokenisation step, binary term-frequency vectorisation, MinHash signature computation, and an approximate similarity join based on the banding technique of LSH. The reference implementation is also extended replacing the bag-of-words strategy with word n-gram shingles, allowing a different kind strategy between the two encodings.

The structure of this report is organised as follows:

- Section 2 describes the arXiv dataset and the preprocessing pipeline.
- Section 3 explain the algorithmic approach, covering the four-stage Spark ML pipeline and the configuration of MinHash and LSH..
- Section 4 outlines the experimental methodology, including the qualitative inspection of similar pairs and the comparison between bag-of-words and n-gram shingling.
- Section 5 presents the empirical results obtained on a 60,000 abstract sub-sample of arXiv.
- Section 6 discusses the scalability of the proposed solution with different levels o N abstract.
- Section 7 concludes the report with key findings.

The primary goal was to build a solution that balances computational efficiency and detection accuracy that can also scale to large amount of data. PySpark helped me in achieving this balance, showing how a modern big-data stack can address a classical similarity search problem at scale.

2 Dataset and Preprocessing

The arXiv dataset was obtained via the Kaggle API[1] and consists of a single newline-delimited JSON file (`arxiv-metadata-oai-snapshot.json`) of approximately 5 GB, containing one record per paper. Each record exposes a set of metadata fields (`id`, `submitter`, `authors`, `title`, `abstract`, `categories`, `update_date`, ...), of which only `id`, `abstract` and `categories` were retained.

The preprocessing pipeline performs the following steps:

1. **Schema reduction**: only `id`, `abstract` and `categories` are selected, reducing memory footprin.
2. **Data filter**: abstracts shorter than 200 characters are discarded as likely-stub records. This guarantees a minimum number of tokens per document and therefore, a significant MinHash signature.
3. **Sub-sampling**: a configurable random sub-sample of 60,000 abstracts is drawn (with fixed seed for reproducibility) for development and experiments, with the possibility to scale to the full dataset by setting a single parameter.

[1] https://www.kaggle.com/datasets/Cornell-University/arxiv

4. **Partitioning and caching**: the resulting DataFrame is repartitioned and cached so that all downstream operations operate in-memory and in parallel.

The text preprocessing happens inside the Spark ML pipeline. Each abstract is split on non-word characters with regex, tokens shorter than three characters are dropped, English stop words are removed, and the remaining tokens are encoded as a binary term-frequency vector. The binary encoding is the natural fit for Jaccard similarit.

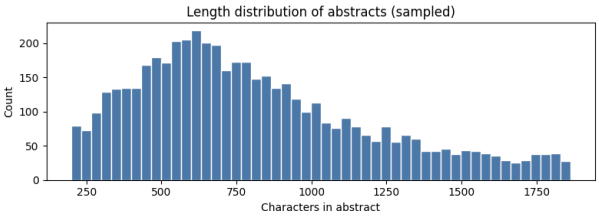

Figure 1: Length distribution of abstract

3 Algorithmic Approach

The algorithm relies on Locality-Sensitive Hashing with MinHash signatures in order to reduce the cost of similarity detection from $O(n^2)$ to approximately $O(n)$, while maintaining a high recall on near-duplicate pairs. Two complementary representations are evaluated: a bag-of-words encoding and a word n-gram shingling variant.

3.1 Text Processing Pipeline

A four-stage Spark ML pipeline transforms each raw abstract into hash signatures suitable for similarity search:

1. **Regex Tokenisation**: splits the abstract on non-word characters, producing a list of tokens of length at least three.
2. **Stop Word Removal**: filters out common English words using Spark's default English stop word list, retaining only content-bearing terms.
3. **Binary Hashing TF**: encodes the token set as a sparse binary vector of dimension $2^{18} = 262{,}144$, where each dimension indicates the presence of a token in the document.
4. **MinHash LSH**: computes the MinHash signatures and the banded LSH index used by the approximate similarity join.

3.2 LSH Configuration and Parameters

The MinHash LSH implementation uses the following hyper-parameters:

- **Number of hash functions** $h = 10$, organised in $b = 10$ **bands** of $r = 1$ rows each. Why b = 10? $P_{\text{cand}}(0.80) = 1 - 0.2^{10} \approx 0.9999998$.
- **Feature space**: $2^{18} = 262\,144$ dimensions in the binary HashingTF vector.
- **Similarity threshold**: Jaccard similarity $s \geq 0.80$, i.e. Jaccard distance ≤ 0.20 in the call to `approxSimilarityJoin`.

3.3 Variant: word n-gram shingling

The default pipeline encodes each abstract as a *bag of single words*, i.e. a set of 1-grams. As an alternative, documents can be represented using *k-shingles*: contiguous sequences of k words, which capture local word order and yield a stricter notion of similarity than unordered vocabulary overlap. A second pipeline is evaluated in which a UDF replaces the filtered word tokens with their word n-grams (with $n = 5$). The downstream stages (`HashingTF` and `MinHashLSH`) are unchanged. The comparison between the two encodings is reported in Section 5.

4 Experimental Methodology

To assess the behaviour of the LSH-based detector two complementary experiments are run on the 60,000 abstract sub-sample of arXiv: (i) a qualitative inspection of the candidate pairs returned by the default pipeline; (ii) a comparison between the bag-of-words encoding and the n-gram shingling variant.

4.1 Qualitative inspection of similar pairs

The first experiment runs the default pipeline (bag-of-words encoding, Jaccard threshold 0.80) and lists the top candidate pairs ranked by estimated Jaccard similarity. Each pair is printed together with the two abstracts so that genuine similarities can be verified by inspection. The expected sources of near-duplication on arXiv are: multiple versions of the same paper (v1, v2, ...), proceedings versions of a preprint, and concurrent submissions to two adjacent categories.

4.2 Comparison of encodings

The second experiment runs the alternative pipeline based on word n-gram shingling. The same threshold and the same MinHash configuration are used, so that the only varying ingredient is the encoding of the documents. The two pipelines are compared on:

- the number of candidate pairs returned at the target similarity threshold;
- the qualitative nature of the top candidate pairs.

The shingling variant is stricter than the bag-of-words encoding, since it requires identical contiguous sequences of words rather than mere common vocabulary.

5 Results

The pipeline was evaluated on a sub-sample of 60 000 arXiv abstracts, all results in this section refer to that working sample.

5.1 Bag-of-words pipeline

At the configured Jaccard threshold $s \geq 0.80$, the default bag-of-words pipeline returns 203 candidate pairs. A manual inspection of the top-ranking pairs reveals that they all correspond to abstracts that are virtually identical at the textual level. Plausible explanations include cross-listings of the same paper, near-simultaneous resubmissions, or administrative duplicates. For example the latter case is directly observable for the pair `0804.1637` – `0804.1638`.

5.2 Comparison with 5-gram shingling

Replacing the bag-of-words encoding with word 5-gram shingles returns 90 pairs at the same threshold. This is the expected behaviour: requiring identical contiguous sequences of five words is a strictly stronger condition than sharing single tokens. The fact that the surviving pairs still reach Jaccard ≈ 1.0 confirms that they correspond to textual duplicates rather than topically related abstracts with overlapping terminology.

6 Scalability

N abstract	Time (s)	Bag of words	5-shingle
3 000	129.6s	6	2
10 000	296.8s	16	8
30 000	1648.6s	96	46
60 000	6099.7s	203	90

Table 1: Computation time and results with different N subsample

The total execution time grows from 130 s on 3 000 abstracts to $\sim$ 6 100 s on 60 000, confirming that the computational time does not follow a linear trend. Instead the number of detected near-duplicates increases super-linearly in both pipelines (BoW: $6 \to 203$, 5-gram: $2 \to 90$).

7 Conclusions

This work implemented a scalable similarity-detection system for the arXiv dataset, based on Apache Spark and Locality-Sensitive Hashing. The system shows how a modern distributed-processing stack combined with the probabilistic guarantees of MinHash can effectively tackle the $O(N^2)$ pairwise comparison bottleneck.

7.1 Key achievements

The approach used was able to successfully discover similar pairs of abstracts while simultaneously reducing the computational cost through LSH. The algorithm is very efficient with a small sample size (3000 / 10000), but if the dataset grows, the time for computation increases non linearly, suggesting that a more powerful machine is needed.

I declare that this material, which I now submit for assessment, is entirely my own work and has not been taken from the work of others, save and to the extent that such work has been cited and acknowledged within the text of my work, and including any code produced using generative AI systems. I understand that plagiarism, collusion, and copying are grave and serious offences in the university and accept the penalties that would be imposed should I engage in plagiarism, collusion or copying. This assignment, or any part of it, has not been previously submitted by me or any other person for assessment on this or any other course of study.